

Week 5 Day 2: Raw Python Agent vs LangChain Agent

Day 1 (agent_gateway.py) was a hand written ReAct loop with two tools. Day 2 rebuilds it in LangChain as a Travel Concierge Bot with four tools (calculator, weather, cost of living CSV, live currency API), conversation memory, and structured output.

Concept Mapping

Day 1 Raw Python	LangChain
OpenAI(base_url, api_key)	ChatOpenAI(base_url, api_key)
Hand written TOOLS JSON schemas	@tool decorated functions
TOOL_FUNCTIONS dict + execute_tool()	Handled inside AgentExecutor
Manual for-loop over steps	AgentExecutor.invoke()
messages list built by hand	RunnableWithMessageHistory
Manual [REASON]/[ACT]/[OBSERVE] prints	verbose=True
try/except around each tool call	handle_tool_error on @tool

Annotated Reasoning Trace

Query: "What's the weather in Tokyo and Reykjavik, and which is warmer right now?"

```
Invoking get_weather(Tokyo) <- ACT
```

```
{temp_c: 31.4, humidity_pct: 80, ...} <- OBSERVE
```

```
Invoking get_weather(Reykjavik) <- ACT
```

```
{temp_c: 10.8, humidity_pct: 95, ...} <- OBSERVE
```

```
"Tokyo is considerably warmer than Reykjavik right now." <- REASON + final answer
```

Two ACT/OBSERVE pairs (one per city), then one REASON step combining both into a comparison, no further tool calls needed.

What LangChain Made Easier

Tool dispatch, JSON argument parsing, and the call-or-finish loop are handled internally instead of a hand written loop. Swapping providers is a one line ChatOpenAI change. Multi-turn memory ('compare it to Y') came from wrapping the executor rather than manually tracking a messages list. Structured final answers (Pydantic model) took a few lines instead of a custom text parser.

Where Complexity Leaked

LangChain 1.x removed create_tool_calling_agent/AgentExecutor entirely, forcing a pin to 0.3.27, real version churn risk. LangChain 0.3.x's Pydantic v1 shim breaks on Python 3.13+/3.14 with an opaque TypeError deep in import internals. A tool reading a pandas DataFrame into json.dumps() failed on int64 not being JSON serializable, a cross-library friction point raw Python with plain dicts would not surface.

handle_tool_error only reliably catches LangChain's own ToolException, not a plain RuntimeError, and must be set as an attribute, not passed as a keyword to @tool. verbose=True shows tool calls and results but not the literal message list being assembled, making some bugs harder to trace than a self written loop.

Real-World Limitations Found

A currency conversion tool (Frankfurter API) covers only ~30 ECB currencies; requesting PKR returned a 404, and the agent retried blindly instead of reporting the limitation, hitting max_iterations. Fix: the tool's error message had to explicitly instruct the model not to retry, proving error-message wording shapes agent behavior as much as the code does. Separately, the gateway's underlying model hit a provider-side quota (429/403), unrelated to the agent's logic, a reminder that shared or free-tier gateways add a failure mode invisible from the application code.

Bottom Line

LangChain removes real boilerplate (tool dispatch, memory bookkeeping) at the cost of framework version risk and less direct debuggability than a self written loop. For a small two tool agent, raw Python was arguably simpler to debug; for a four tool agent with memory and structured output, LangChain's abstractions paid for themselves despite the setup friction documented above.